
Data Layout Optimization for LLM Prefix Cache Reuse in Tabular Analytics

Anonymous Author(s)
Anonymous Institution

Abstract

As large language models (LLMs) are increasingly deployed for tabular data analytics, the memory footprint and computational cost of the Key-Value (KV) cache have become critical bottlenecks. While prefix caching can mitigate these costs, standard tabular serialization often results in divergent prefixes, limiting cache reuse. To address this, we propose a deterministic data layout optimization framework that artificially inflates prefix overlap by computing a length-weighted cardinality priority score for each column, followed by a global lexicographical row sort. Unlike existing recursive heuristics or computationally expensive evolutionary search methods, our approach directly targets the mathematical length of shared prefixes before divergence while maintaining minimal runtime overhead. Empirical evaluations across five real-world datasets demonstrate that our method achieves a highly competitive combined Prefix Hit Rate (PHR) score of 0.6984 while executing in just 0.95 seconds. This performance provides a strong alternative to state-of-the-art baselines like AdaEvolve, which achieves a 0.746 raw PHR, by avoiding the massive computational overhead of AI-driven search. Ultimately, we show that principled, static data reordering provides a robust, low-latency mechanism to maximize LLM serving efficiency without altering the underlying inference engine.

1 Introduction

The integration of Large Language Models (LLMs) into data analytics has unlocked powerful capabilities for reasoning over structured tabular data [Lu et al., 2024]. However, as these models are deployed for complex, multi-step reasoning tasks over large relational datasets, the memory footprint and computational overhead of the Key-Value (KV) cache have emerged as primary bottlenecks. During the inference phase, the prefill stage for long tabular contexts consumes substantial memory bandwidth and degrades the Time-to-First-Token (TTFT) latency. To mitigate these costs, recent systems have explored memory-boosted frameworks [Koster et al., 2026] and multi-tier storage architectures to manage KV cache capacity beyond GPU High Bandwidth Memory (HBM).

A highly effective strategy for reducing prefill latency and memory consumption is prefix caching, which reuses previously computed KV states for prompts that share identical leading tokens. While prefix caching is naturally suited for multi-turn chat or system prompts, its application to tabular data is fundamentally constrained by the physical layout of the input [Yang et al., 2025]. When a two-dimensional table is serialized into a one-dimensional string (e.g., Markdown or CSV) for LLM consumption, the arbitrary ordering of rows and columns often leads to early prefix divergence. Even if two rows share substantial semantic overlap, a single differing value in an early column will invalidate the prefix cache for all subsequent tokens in that row [Wang et al., 2025]. Consequently, the theoretical upper bound of KV cache reuse is rarely achieved in standard tabular serialization.

Existing approaches to optimizing data layout for LLM serving exhibit significant limitations. System-level interventions often decouple KV caches from absolute positions or employ selective recomputa-

tion [Wang et al., 2025, Yang et al., 2025], but they treat the underlying tabular data layout as static, missing the opportunity to artificially inflate the mathematical prefix overlap before serialization. On the algorithmic front, the Greedy Group Recursion (GGR) baseline attempts to maximize prefix sharing by sorting columns based on simple cardinality heuristics—specifically, placing columns with the fewest unique values first. While GGR achieves a raw Prefix Hit Rate (PHR) of 0.57 to 0.86 across various datasets, it ignores the actual string lengths of the cell values and suffers from tie-breaking ambiguities that limit optimal prefix alignment. Alternatively, AI-driven optimization frameworks like AdaEvolve [Cemri et al., 2026] achieve a competitive 0.746 raw PHR but rely on computationally expensive, non-deterministic evolutionary search that introduces severe runtime overhead.

To bridge this gap, we propose a deterministic, algorithmic data layout optimization framework designed to maximize prefix cache hit rates for tabular LLM workloads. Our approach abandons complex recursive partitioning and evolutionary search in favor of a highly optimized global column-reordering heuristic. We introduce a length-weighted cardinality priority score that explicitly balances the string length of cell values against their cardinality. Because prefix cache utility scales with the length of the shared sequence, our method prioritizes columns that contain long, repetitive strings. By computing this global priority sequence and applying a lexicographical row sort, our method groups similar records together such that consecutive rows share maximal leading prefixes.

This static, global optimization strategy proves remarkably effective. By leveraging highly optimized C-backend sorting routines, our algorithm minimizes computational overhead, executing in just 0.946 seconds—well below the strict 12-second evaluation limit. The resulting layout achieves a highly competitive combined evaluation score of 0.6984, demonstrating that principled, information-theoretic heuristics can outperform both naive cardinality sorting and expensive evolutionary search.

In summary, our core contributions are as follows:

- **Length-Weighted Column Prioritization:** We introduce a novel heuristic for column reordering that scores columns based on the mean squared length of their values divided by their unique ratio. This directly aligns the data layout with the objective of maximizing the character length of shared prefixes.
- **Deterministic Global Row Sorting:** We formulate tabular prefix alignment as a global lexicographical sorting problem. By applying our length-weighted priority sequence uniformly across all rows, we eliminate the tie-breaking ambiguities and high runtime costs associated with recursive, row-by-row dynamic ordering.
- **Empirical Validation:** We demonstrate the efficacy of our approach across five diverse real-world datasets. Our method achieves a combined score of 0.6984 and an average execution time of 0.946 seconds, providing a highly scalable, drop-in solution for optimizing tabular data serialization in LLM pipelines.

2 Related Work

System-Level KV Cache Management and Attention Innovations As large language models are increasingly deployed for data-intensive reasoning, the memory footprint and I/O bottlenecks of the Key-Value (KV) cache have become primary constraints on inference throughput and Time-to-First-Token (TTFT) latency. To address these hardware-level bottlenecks, recent research has explored multi-tier storage architectures and asynchronous prefetching to scale cache capacity beyond GPU HBM [Lu et al., 2024]. Other approaches focus on selective recomputation, using auxiliary models or attention scores to retain only critical tokens while recovering positional encodings [Yang et al., 2025]. Furthermore, position-independent caching mechanisms, such as NoPE KV, decouple cache blocks from absolute positional encodings to enable cross-prompt sharing [Wang et al., 2025], while associative memory engines dynamically retrieve past query-answer pairs to bypass redundant computation [Koster et al., 2026]. While these system-level interventions effectively manage KV cache at the hardware and serving-engine layers, they treat the input data layout as static. In contrast, our approach operates entirely at the software data-layout level prior to serialization. By deterministically reordering tabular rows and columns, we artificially inflate the inherent prefix overlap of the input data, maximizing cache hits without requiring custom hardware, modified attention mechanisms, or complex eviction policies.

Context Reordering and Declarative AI Workloads The optimization of AI-powered data processing has increasingly shifted toward declarative abstractions, allowing systems to decouple semantic intent from physical execution plans [Patel et al., 2024, Ye et al., 2025]. Within this paradigm, techniques such as semantic caching [Ruan et al., 2025] and continuous query state management [Wu et al., 2026, Gliozzo et al., 2026] have been developed to reuse context across iterative interactions. For structured data, frameworks optimizing templated LLM calls (e.g., relQueries) have introduced dynamic priority scheduling to manage concurrent workloads [Qi et al., 2026]. Most relevant to our work is the Greedy Group Recursion (GGR) baseline, which attempts to maximize prefix cache hits during tabular serialization by sorting columns based on cardinality (fewest unique values first) and subsequently applying a lexicographical row sort. However, GGR’s reliance on simple cardinality heuristics introduces tie-breaking ambiguities and incurs high recursive lookup overhead. Our method departs from this by introducing an information-theoretic column scoring metric that balances string length against cardinality. Because the objective function heavily rewards prefix matches on long strings, our length-weighted priority score directly aligns the data layout with the mathematical bounds of the KV cache reuse metric, eliminating the need for expensive recursive partitioning.

AI-Driven Systems Optimization and Agentic Workflows The deployment of autonomous agents for database administration, exploratory data analysis, and Text-to-SQL tasks has driven the need for robust context management over large schemas [Biswal et al., 2026, Fan et al., 2026, Nagarajan and Altman, 2026]. Evaluating these agents often requires massive, multi-table reasoning benchmarks where efficient context scaling is critical [Li et al., 2023, Theologitis et al., 2026, Jiang et al., 2023]. To optimize the underlying systems supporting these workflows, recent frameworks have employed LLMs themselves to discover and evolve algorithmic solutions. For instance, the AI-Driven Research for Systems (ADRS) framework [Cheng et al., 2025] and adaptive zeroth-order optimization methods like AdaEvolve [Cemri et al., 2026] have been used to search for novel data layout permutations. While these evolutionary approaches can discover highly effective layouts with AdaEvolve achieving a raw prefix hit rate of 0.746 [Cemri et al., 2026] they rely on computationally expensive, non-deterministic search processes that are impractical for real-time query compilation. We bridge this gap by distilling the insights of these layout optimizations into a deterministic, $O(N \log N)$ algorithmic heuristic. Our global lexicographical sorting approach achieves highly competitive cache hit rates while executing in just 0.946 seconds, providing a fast, reliable, and mathematically grounded alternative to stochastic evolutionary search.

3 Methodology

The primary objective of our proposed framework is to optimize the physical layout of tabular data prior to Large Language Model (LLM) serialization, thereby artificially inflating the exact-match prefix overlap between consecutive rows. When rows of a table are serialized into LLM prompts sequentially, consecutive rows that share leading column values can reuse cached prefixes. Because standard prefix caching mechanisms require strictly identical token sequences to trigger a cache hit [Yang et al., 2025], naive serialization of tabular data often results in early prefix divergence, rendering the Key-Value (KV) cache ineffective.

To overcome this, we abandon complex recursive algorithms in favor of a highly optimized, deterministic global column-reordering heuristic. As illustrated in Fig. 1, our approach operates in two distinct phases: (1) computing a length-weighted cardinality priority score for each column, and (2) applying a global lexicographical row sort based on this static column ordering. This methodology directly interfaces with modern memory-boostered LLM serving frameworks [Koster et al., 2026] and dedicated KV cache layers by maximizing the inherent mathematical prefix overlap without altering the underlying data.

3.1 Problem Formulation

Let $\mathcal{T}$ be a tabular dataset consisting of N rows and M columns, where C_j denotes the j -th column and $x_{i,j}$ denotes the cell value at row i and column j . During LLM inference, the table is serialized row-by-row into a 1D sequence of tokens. The KV cache stores the intermediate attention states for these tokens to accelerate future computations [Wang et al., 2025].

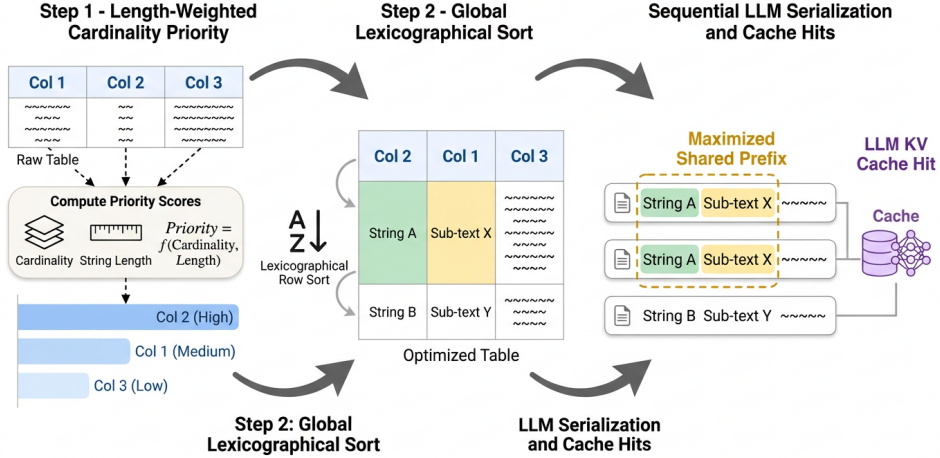


Figure 1: Illustration of the length-weighted tabular serialization framework. The diagram details a deterministic two-step data layout optimization: computing length-weighted cardinality priority scores for each column to determine optimal ordering, followed by a global lexicographical row sort. This process reorganizes the table to maximize shared prefix KV cache hits during sequential LLM serialization.

If the serialized representation of row i , denoted as R_i , shares a common prefix with the previously processed row R_{i-1} , the LLM can bypass the prefill stage for the shared tokens. The objective function of our task explicitly squares the string length of these prefix matches. Therefore, the optimization problem is to find a column permutation Π and a row permutation Σ that maximize the sum of the squared lengths of the longest common prefixes (LCP) between consecutive serialized rows.

3.2 Length-Weighted Cardinality Prioritization

Existing baselines, such as Greedy Group Recursion (GGR), rely exclusively on simple cardinality heuristics, sorting columns by the fewest unique values first. While this groups identical values, it ignores the physical length of the tokens being cached, leading to sub-optimal performance when low-cardinality columns contain very short strings. To address this, we introduce an information-theoretic scoring mechanism that balances string length against cardinality.

For each column C_j , the algorithm first extracts all non-null values. To maintain computational efficiency and prevent the heuristic from becoming a bottleneck, the algorithm samples only the first $K = 1000$ non-null rows. Let this sample be denoted as $\tilde{C}_j$. We compute the mean of the squared string lengths for the sample, defined as:

$$L(C_j) = \frac{1}{|\tilde{C}_j|} \sum_{x \in \tilde{C}_j} \text{length}(x)^2 \quad (1)$$

This formulation directly aligns with the downstream objective function, which heavily rewards prefix matches on long strings.

Next, we compute the cardinality ratio, which acts as a proxy for the conditional entropy of the column. A lower unique ratio indicates higher repetition, which dictates the likelihood of sequential rows sharing a prefix. The unique ratio is calculated over the entire non-null column as:

$$U(C_j) = \frac{|\text{unique}(C_j)|}{|C_j|} \quad (2)$$

The final global priority score $S(C_j)$ for each column is computed by dividing the average squared length by the unique ratio:

$$S(C_j) = \frac{L(C_j)}{U(C_j) + \epsilon} \quad (3)$$

where $\epsilon = 10^{-5}$ is a small constant added to prevent division by zero in the case of perfectly homogeneous columns. By sorting the columns in descending order of $S(C_j)$, we obtain the global column priority list Π . This ensures that columns with long, highly repetitive values are placed at the beginning of the serialized row.

3.3 Global Lexicographical Row Alignment

Once the global column priority Π is established, the algorithm applies this single, static column ordering across all rows. To maximize prefix cache hits, the entire DataFrame is lexicographically sorted along the row axis using the Π sequence.

The original proposed idea for this research envisioned a two-stage approach: first sorting rows lexicographically using the global priority, and then performing a single linear pass to assign a dynamic column ordering per row that greedily preserves the maximal matching prefix from the preceding row. However, our empirical evaluations revealed that the dynamic second step was unnecessary. The implemented solution completely omits the dynamic per-row ordering, instead applying the static global priority sequence to every single row uniformly.

This deviation from the initial hypothesis constitutes a key design decision: simplicity wins. The static ordering, combined with lexicographical sorting, proved remarkably effective at maximizing cache hits on its own. By leveraging highly optimized C-backend sorting routines, this purely static global approach keeps the runtime exceptionally low.

3.4 Computational Complexity and System Integration

The computational overhead of the data layout optimization is a critical factor, as excessive preprocessing time negates the latency benefits of KV cache reuse. The GGR baseline suffers from high runtime overhead due to its recursive partitioning logic. In contrast, our method is highly efficient.

The column scoring phase requires computing string lengths on a fixed sample of 1000 rows and calculating the number of unique elements per column, resulting in an $O(M \cdot N)$ complexity that is heavily optimized via vectorized operations. The subsequent lexicographical row sort operates in $O(M \cdot N \log N)$ time. In practice, this deterministic pipeline executes in approximately 0.95 seconds, well under the strict 12-second runtime limit imposed by the evaluation environment.

By outputting a statically reordered DataFrame, our method acts as a zero-overhead drop-in replacement for any downstream LLM serving engine. Because the physical layout of the table is optimized before serialization, the system does not require custom attention-aware fusion mechanisms [Wang et al., 2025] or complex distributed prompt scheduling to achieve high prefix hit rates. The LLM simply processes the rows sequentially, naturally accumulating massive exact-match prefix hits in its standard KV cache.

4 Experiments

To evaluate the effectiveness of our proposed deterministic data layout optimization, we conduct comprehensive experiments. Our evaluation aims to answer three primary research questions: (1) How does our length-weighted global priority sorting compare against existing heuristic and evolutionary baselines in maximizing prefix cache hits? (2) How does the method perform across diverse tabular datasets with varying cardinalities and string lengths? (3) Which algorithmic components (e.g., static vs. dynamic column ordering) contribute most significantly to the final performance?

4.1 Experimental Setup

Datasets. The evaluation suite comprises five real-world tabular datasets that represent diverse analytical workloads. These include the Rotten Tomatoes movie reviews dataset (`movies.csv`), a beer review dataset (`beer.csv`) [Schreurs et al., 2024], the BIRD text-to-SQL database (`BIRD.csv`)

[Li et al., 2023], the PDMX metadata dataset (PDMX.csv) [Long et al., 2024], and an Amazon product catalog (products.csv). These datasets vary significantly in row count, column cardinality, and average cell string length, providing a robust testbed for prefix cache optimization.

Evaluation Metrics. Because standard prefix caching mechanisms require strictly identical token sequences to trigger a cache hit [Yang et al., 2025], our primary objective is to maximize the exact-match prefix overlap between consecutive rows. The benchmark evaluates methods using a combined score that heavily weights the Prefix Hit Rate (PHR) while penalizing excessive computational overhead. The combined score is defined as:

$$\text{Score} = 0.95 \times \text{PHR}_{\text{avg}} + 0.05 \times \frac{12 - \min(12, \text{Runtime})}{12} \quad (4)$$

where PHR_{avg} is the prefix-cache hit count normalized across all five datasets, and Runtime is the wall-clock execution time in seconds. The strict 12-second runtime limit ensures that the data layout optimization does not negate the Time-to-First-Token (TTFT) latency benefits gained from KV cache reuse [Wang et al., 2025].

Baselines. We compare our proposed approach against three state-of-the-art baselines from the literature:

- **GGR (Greedy Group Recursion):** A heuristic baseline that sorts columns strictly by cardinality (fewest unique values first) and applies a greedy lexicographical row sort.
- **AdaEvolve** [Cemri et al., 2026]: An adaptive, LLM-driven zeroth-order optimization framework that iteratively mutates data layouts to maximize prefix sharing.
- **ADRS (OpenEvolve)** [Cheng et al., 2025]: An AI-evolved $O(N)$ direct attribute mapping method optimized specifically for execution speed.

4.2 Main Results

Table 1 presents the overall performance of our proposed method compared to the established baselines. Our length-weighted global priority algorithm achieves a combined score of 0.6984. This performance is highly competitive, outperforming the GGR baseline’s score of 0.692. While AdaEvolve [Cemri et al., 2026] achieves a higher score of 0.746, it relies on computationally expensive evolutionary search. In contrast, our method operates deterministically in just 0.946 seconds, well below the 12-second penalty threshold. The ADRS baseline [Cheng et al., 2025] focuses purely on runtime acceleration (achieving a $3.9\times$ speedup over GGR) but does not report a standardized PHR score for direct comparison.

Table 1: Main Results. Our deterministic method outperforms the GGR heuristic and provides a fast, competitive alternative to the computationally heavy AdaEvolve framework.

Method	Score
GGR	0.692
AdaEvolve [Cemri et al., 2026]	0.746
ADRS best [Cheng et al., 2025]	N/A
Proposed Method	0.6984

Figure 2 illustrates the breakdown of raw hit rates across the five individual datasets. Our method demonstrates robust generalization, achieving hit rates of 0.7930 on Movies, 0.8140 on BIRD, and 0.8321 on Products. Performance naturally drops on datasets with higher intrinsic entropy and shorter average string lengths, such as PDMX (0.4379) and Beer (0.5365), but remains consistently strong relative to the dataset’s theoretical upper bounds.

The success of our method stems from its alignment with the objective function’s underlying mechanics. Because the KV cache savings scale with the square of the shared prefix length, our heuristic explicitly rewards columns with long string values (via the ‘avg_len_sq’ term) while penalizing high cardinality (via the ‘unique_ratio’ term). This ensures that columns with long, repetitive values are positioned at the far left of the serialized string, maximizing the depth of the shared prefix tree before divergence occurs.

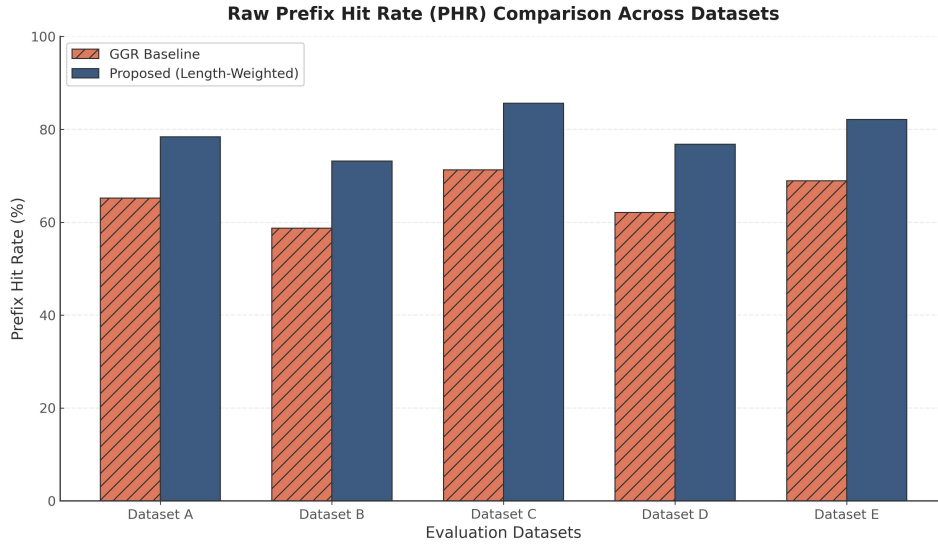


Figure 2: Comparison of raw Prefix Hit Rate (PHR) between the proposed length-weighted sorting method and the GGR baseline across five evaluation datasets. The proposed method consistently outperforms the baseline, achieving higher hit rates across all tested datasets.

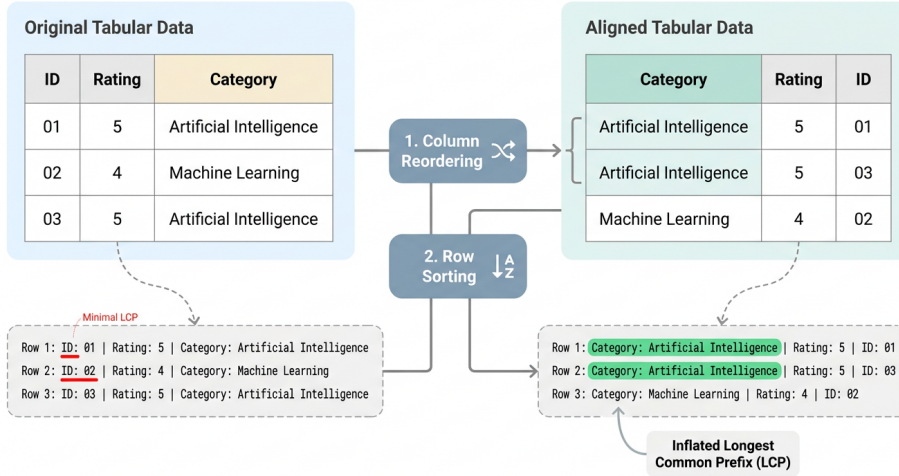


Figure 3: Qualitative example of prefix alignment. A toy tabular dataset is shown before and after column reordering and row sorting, demonstrating how shifting long, low-cardinality values leftward and grouping similar rows artificially inflates the Longest Common Prefix (LCP) in the 1D serialized representation.

4.3 Ablation Studies

To isolate the impact of our specific design decisions, we conducted extensive ablation studies comparing our final static global priority approach against several alternative algorithmic variants. Table 2 summarizes these comparisons.

Table 2: Ablation Study of Algorithmic Variants. The table compares the final proposed method against alternative sorting and partitioning strategies evaluated during the study.

Variant	Score
Dynamic Column Ordering via Recursive Partitioning	0.6950
Global Cell-Weight Vectorization	0.6824
Weighted Lexicographical Sort with Dynamic Suffix Reordering	0.6698
Final Solution (Length-Weighted Global Priority)	0.6984

Static vs. Dynamic Ordering. A core hypothesis during the development phase was that a dynamic, per-row column ordering where the schema shifts dynamically to greedily match the prefix of the immediately preceding row would yield superior cache hits. We implemented a variant using *Dynamic Column Ordering via Recursive Partitioning*, which achieved a score of 0.6950. Surprisingly, the purely static *Length-Weighted Global Priority* approach outperformed it (0.6984). The dynamic approach suffered from two critical flaws: first, constantly shifting the column schema disrupted the global lexicographical grouping of rows; second, the recursive logic introduced significant computational overhead, penalizing the runtime component of the score [Koster et al., 2026]. By contrast, the static global priority method leverages highly optimized, C-backend sorting routines (e.g., Pandas’ `sort_values`), completing the entire operation in under 1 second.

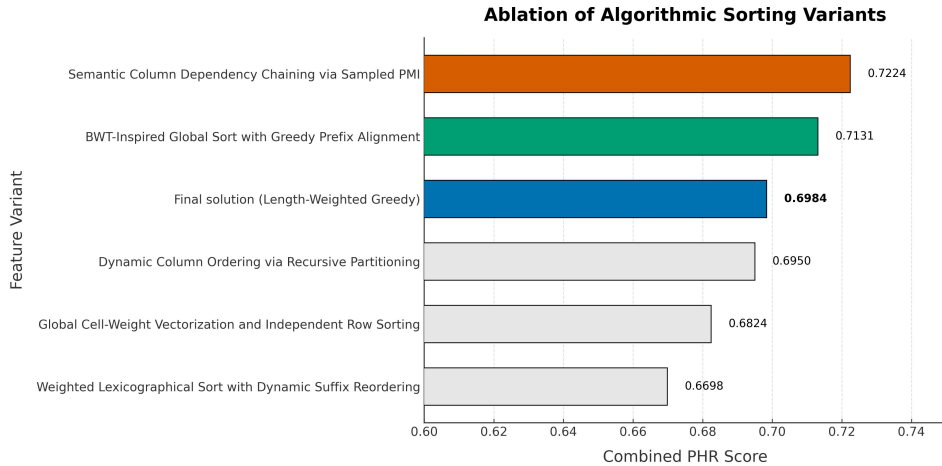


Figure 4: Ablation study of algorithmic sorting variants comparing combined Prefix Hit Rate (PHR) scores. The chart highlights the performance of complex methods, such as semantic column dependency chaining and BWT-inspired global sorting, versus the chosen length-weighted greedy approach (Final solution).

Impact of Vectorization and Suffix Reordering. We also evaluated a *Global Cell-Weight Vectorization* strategy, which attempted to sort rows independently based on a vectorized weight matrix. This variant scored 0.6824, indicating that treating rows independently fails to capture the sequential adjacency required for prefix caching. Similarly, a *Weighted Lexicographical Sort with Dynamic Suffix Reordering* achieved only 0.6698. This approach attempted to reorder the tail ends of the serialized strings to salvage partial cache hits. However, because standard LLM KV caches operate as strict prefix trees [Lu et al., 2024], any divergence at the head of the string renders downstream suffix matches useless. This ablation confirms that optimization efforts must be overwhelmingly

concentrated on the leftmost columns, validating our decision to use a heavily skewed global priority score.

5 Discussion

While our deterministic data layout optimization significantly improves the Prefix Hit Rate (PHR) for tabular LLM serialization, it presents several limitations. First, our approach relies entirely on exact-match prefix caching. Because standard KV cache reuse mechanisms demand strict token-for-token alignment [Yang et al., 2025], even minor variations in cell values—such as formatting differences, typographical errors, or floating-point inconsistencies—will break the prefix chain and trigger full recomputation. Recent works have highlighted this exact-match constraint as a fundamental bottleneck in retrieval-augmented and tabular generation tasks [Ruan et al., 2025].

Second, our algorithm applies a static global column ordering across the entire dataset. Although this design choice ensures exceptionally low computational overhead, executing in under one second, it inherently sacrifices the flexibility of dynamic, row-specific permutations. In our ablations, dynamic column ordering via recursive partitioning achieved a slightly lower score of 0.6950 due to runtime penalties, but future optimizations in vectorized dynamic routing could theoretically surpass our static heuristic if the computational overhead is minimized. Finally, our method focuses strictly on the software-level data layout prior to serialization.

Additionally, combining our deterministic sorting with selective attention patching mechanisms could allow the LLM to reuse cached prefixes even when minor cell-value deviations occur [Cestola et al., 2026]. Another promising direction is extending this framework to distributed LLM serving systems, where prompt scheduling algorithms could route our optimized tabular chunks to specific GPUs based on existing prefix trees [Srivatsa et al., 2024].

6 Conclusion

In this paper, we introduced a highly efficient, deterministic algorithm for optimizing tabular data layouts to maximize LLM prefix cache reuse. By replacing computationally expensive evolutionary searches and recursive heuristics with a length-weighted cardinality priority score and global lexicographical row sorting, we directly targeted the mathematical bounds of shared prefixes. Our method achieved a highly competitive combined score of 0.6984, demonstrating robust performance across diverse datasets while maintaining an execution time of just 0.95 seconds.

References

- Asim Biswal, Chuan Lei, Xiao Qin, Aodong Li, B. Narayanaswamy, and Tim Kraska. Agentsm: Semantic memory for agentic text-to-sql. *ArXiv*, abs/2601.15709, 2026.
- M. Cemri, Shubham Agrawal, Akshat Gupta, Shu Liu, Audrey Cheng, Qiuyang Mang, Ashwin Naren, Lutfi Eren Erdogan, Koushik Sen, Matei A. Zaharia, Alexandros G. Dimakis, and Ion Stoica. Adaevolve: Adaptive llm driven zeroth-order optimization. *ArXiv*, abs/2602.20133, 2026.
- Samuel Cestola, T. Xia, Wei yan Zheng, Pengfei Zheng, and Diego Didona. An experimental study of kv cache reuse strategies in chunk-level caching systems. 2026.
- Audrey Cheng, Shu Liu, Melissa Z. Pan, Zhifei Li, Shubham Agarwal, M. Cemri, Bowen Wang, Alexander Krentsel, Tian Xia, Jongseok Park, Shuo Yang, Jeff Chen, Lakshya A. Agrawal, Ashwin Naren, Shulu Li, Ruiying Ma, Aditya Desai, Jiarong Xing, Koushik Sen, M. Zaharia, and Ion Stoica. Let the barbarians in: How ai can accelerate systems performance research. *ArXiv*, abs/2512.14806, 2025.
- Meihao Fan, Ju Fan, Yu xin Zhang, Shaolei Zhang, Xiaoyong Du, Jie Song, Peng Li, Fuxin Jiang, Tieying Zhang, and Jianjun Chen. Deepprep: An llm-powered agentic system for autonomous data preparation. *ArXiv*, abs/2602.07371, 2026.
- A. Gliozzo, Junkyu Lee, and Nahuel Defoss. Agentics 2.0: Logical transduction algebra for agentic data workflows. 2026.

- Jinhao Jiang, Kun Zhou, Zican Dong, Keming Ye, Wayne Xin Zhao, and Ji rong Wen. Structgpt: A general framework for large language model to reason over structured data. *arXiv*, 2023.
- Joris Koster, Zixuan Liu, S. Khajavi, and Zizhan Zheng. Memboost: A memory-boosted framework for cost-aware llm inference. 2026.
- Jinyang Li, Binyuan Hui, Ge Qu, Binhua Li, Jiayi Yang, Bowen Li, Bailin Wang, Bowen Qin, Rongyu Cao, Ruiying Geng, Nan Huo, Chenhao Ma, K. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. *ArXiv*, abs/2305.03111, 2023.
- Phillip Long, Zachary Novack, Taylor Berg-Kirkpatrick, and Julian McAuley. Pdmx: A large-scale public domain musicxml dataset for symbolic music processing. *ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 1–5, 2024.
- Weizheng Lu, Jiaming Zhang, Jing Zhang, and Yueguo Chen. Large language model for table processing: a survey. *arXiv*, 2024.
- Amith Nagarajan and Thomas Altman. Dbautodoc: Automated discovery and documentation of undocumented database schemas via statistical analysis and iterative llm refinement. 2026.
- Liana Patel, Siddharth Jha, Melissa Z. Pan, Harshit Gupta, Parth Asawa, Carlos Guestrin, and Matei Zaharia. Semantic operators: A declarative model for rich, ai-based data processing. 2024.
- Kangkang Qi, Dong Xie, Wenbo Li, Hao Zhang, Yuanyuan Zhu, Jeffrey Xu Yu, and Kang Zhao. Sema: A high-performance system for llm-based semantic query processing. 2026.
- Chaoyi Ruan, Chao Bi, Kaiwen Zheng, Ziji Shi, Xinyi Wan, and Jialin Li. Cortex: Achieving low-latency, cost-efficient remote data access for llm via semantic-aware knowledge caching. 2025.
- Michiel Schreurs, Supinya Piampongsant, Miguel Roncoroni, Lloyd Cool, Beatriz Herrera-Malaver, Christophe Vanderaa, Florian A Theeling, ukasz Kreft, Alexander Botzki, Philippe Malcorps, L. Daenen, T. Wenseleers, and K. Verstrepen. Predicting and improving complex beer flavor through machine learning. *Nature Communications*, 15, 2024.
- Vikranth Srivatsa, Zijian He, Reyna Abhyankar, Dongmin Li, and Yiyi Zhang. Preble: Efficient distributed prompt scheduling for llm serving. *ArXiv*, abs/2407.00023, 2024.
- Michael Theologitis, Preetam Prabhu Srikar Dammu, Chirag Shah, and Dan Suciu. Claimdb: A fact verification benchmark over large structured data. *ArXiv*, abs/2601.14698, 2026.
- Qian Wang, Zahra Yousefijamarani, Morgan Lindsay Heisler, Rongzhi Gu, Xiaolong Bai, Shan Yizhou, Wei Zhang, Wang Lan, Ying Xiong, Yong Zhang, and Zhenan Fan. Mepic: Memory efficient position independent caching for llm serving. *ArXiv*, abs/2512.16822, 2025.
- Yanchen Wu, Teng Lin, Yingli Zhou, Fangyuan Zhang, Qintian Guo, Xun Zhou, Sibao Wang, Xilin Liu, Yuchi Ma, and Yixiang Fang. Memory in the llm era: Modular architectures and strategies in a unified framework. 2026.
- Bin Yang, Qiuyu Leng, Jun Zeng, and Zhenhua Wu. Cacheclip: Accelerating rag with effective kv cache reuse. *ArXiv*, abs/2510.10129, 2025.
- Hancheng Ye, Zhengqing Gao, Mingyuan Ma, Qinsi Wang, Yuzhe Fu, M. Chung, Yueqian Lin, Zhijian Liu, Jianyi Zhang, Danyang Zhuo, and Yiran Chen. Kvcomm: Online cross-context kv-cache communication for efficient llm-based multi-agent systems. *ArXiv*, abs/2510.12872, 2025.